

AV2 — Teoria da Computabilidade

λ -Cálculo · MT Não Determinística

Turma CC5NA — Prof. Daniel Leal Souza · 01/2026

Equipe: Yuri Aguiar · João Rath · Pedro Paulo

Modelos escolhidos (Seção 3):

- Opção 7 — λ -Cálculo $\rightarrow$ computar o **fatorial** via combinador Y
- Opção 4 — MT Não Determinística $\rightarrow$ reconhecer $L = \{ ww : w \in \{a,b\}^+ \}$

Roteiro (12–15 min) e divisão de falas

Bloco	Tempo	Responsável
Teoria (ambos os modelos)	~3 min	Yuri
Demonstração λ -cálculo	~3 min	João
Demonstração MTND	~3 min	Pedro
Análise, computabilidade e limites	~3 min	Yuri
Arguição	~3 min	todos

Os dois problemas são **distintos**: um *computa uma função* (fatorial); o outro *decide uma linguagem* ($\{ww\}$). Formalismos diferentes, tarefas diferentes.

Parte I — λ -Cálculo (teoria)

Sintaxe (definição indutiva de **termo**):

$$M ::= x \mid (\lambda x. M) \mid (M N)$$

variável | abstração | aplicação

- Variáveis livres $\times$ ligadas; α -equivalência ($\lambda x. x \equiv \lambda y. y$).
- Única regra de cálculo — β -redução: $(\lambda x. M) N \rightarrow M[x := N]$.
- Substituição com captura evitada (α -renomeação de variáveis ligadas).
- Church-Turing: λ -cálculo é Turing-completo (Kleene, 1936).

λ -Cálculo — formalização da implementação

- Numerais de Church: $n = \lambda f\ x. f\ (f\ \dots\ (f\ x))$.
- Booleanos: $\text{TRUE} = \lambda x\ y. x$, $\text{FALSE} = \lambda x\ y. y$, $\text{IF } b\ t\ e = b\ t\ e$.
- Aritmética: $\text{MULT} = \lambda m\ n\ f. m\ (n\ f)$, PRED via pares.
- Recursão (o cálculo puro não tem!): **combinador Y**
 $Y = \lambda f. (\lambda x. f\ (x\ x))\ (\lambda x. f\ (x\ x))$, com $Y\ g = g\ (Y\ g)$.

Problema-alvo:

```
FACT = Y (λf n. IF (ISZERO n) 1 (MULT n (f (PRED n))))
```

Estratégia: β -redução em **ordem normal** (leftmost-outermost) $\rightarrow$ encontra a forma normal se ela existir (Teorema da Padronização).

λ -Cálculo — demonstração (ao vivo)

```
python implementacoes/lambda_calculo/exemplos.py
```

`MULT 2 3` $\rightarrow$ 6 em 7 β -reduções (traço completo):

```
[0] ((\m n f. (m (n f))) 2 3)
...
[7] (\f x. (f (f (f (f (f (f x))))))) = numeral 6
```

Entrada	Resultado	β -reduções
<code>FACT 2</code>	2	248
<code>FACT 3</code>	6	1525
<code>FACT 4</code>	24	10 384

λ -Cálculo — limite e indecidibilidade

$\Omega = (\lambda x. x x) (\lambda x. x x) \rightarrow \Omega \rightarrow \Omega \rightarrow \dots$ (NUNCA para)

- Nem todo termo tem **forma normal** $\rightarrow$ usamos um **limite de passos**.
- Isso reflete a **indecidibilidade do problema da parada**: não há avaliador que decida, para todo termo, se a redução termina.
- Validado por testes: `test_omega_nao_normaliza`, `test_captura_evitada`.

Parte II — MT Não Determinística (teoria)

$M = (Q, \Sigma, \Gamma, \delta, q_0, b, F)$ com a transição como **relação**:

$$\delta : (Q \times \Gamma) \rightarrow P(Q \times \Gamma \times \{L, R, S\})$$

- Um mesmo (estado, símbolo) pode ter **vários** destinos → **árvore** de computação (não um caminho).
- **Aceitação por existência de ramo aceitante.**
- **MTND $\equiv$ MT determinística** (mesma classe de linguagens; custo pode ser exponencial).

MTND — problema e formalização

$L = \{ ww : w \in \{a,b\}^+ \}$ — palavras duplicadas (não é palíndromo!).

- $|Q| = 10 (> 8)$, 37 transições (≥ 10).
- $\Gamma = \{ a, b, A, B, Sa, Sb, C, _ \}$ — marcadores de 1ª/2ª metade.
- **Não determinismo:** em `qScan`, a máquina **adivinha o ponto médio** (continuar na 1ª metade *ou* começar a 2ª aqui).
- **Verificação determinística:** casa 1ª e 2ª metades **na mesma ordem**, marcando símbolos; aceita se ambas esgotam **juntas**.

$L = \{ww\}$ **NÃO** é livre de contexto → um autômato de pilha não basta; precisa do poder da Máquina de Turing.

MTND — demonstração (ao vivo)

```
python implementacoes/mt_nao_deterministica/maquina_ww.py
```

Árvore de computação para **ab** (**rejeita** — todos os ramos morrem):

```
q0: [a] b
├─ qScan: a [b]
│   └─ qScan: a b [_] X (não tagueou → morre)
│       └─ qFindLeft: [a] Sb
│           └─ ... qToBoundaryA: A [Sb] X (a ≠ b → morre)
```

Entrada	Veredito	Entrada	Veredito
aa , abab , aabaab	ACEITA	abba , aba , abc	rejeita

MTND — ramo aceitante de **abab**

```
q0:[a]bab → qScan:a[b]ab → qScan:ab[a]b    (adivinha meio após "ab")  
→ marca 1ª metade e casa com a 2ª, na ordem:  
    qToBoundaryA: A[b]Sa b    (casa 'a' ↔ tag 'a')  
    qToBoundaryB: A B[C]b     (casa 'b' ↔ 'b')  
→ qCheckSecond: A B C C[_] → qAccept ✓
```

Oráculo de verificação: comparamos a MTND com a definição de L para **todas** as 2046 cadeias de $\{a,b\}^*$ até $|w| = 10 \rightarrow 0$ divergências. $L(M) = \{ww\}$.

Análise: computabilidade e limites

- **Church-Turing:** λ -cálculo $\equiv$ MT $\equiv$ funções recursivas. Tudo "efetivamente computável" cai numa dessas formulações.
- **Não determinismo** (MTND) é **conveniência**, não poder extra: adivinhar-e-verificar; toda MTND é simulável por MT determinística.
- **Hierarquia:** $\{ww\}$ separa **livres de contexto** (autômato de pilha) das linguagens que **exigem** uma MT.
- **Limites:** parada **indecidível** (Ω no λ -cálculo; ramos não-parantes no caso geral de MTs) $\rightarrow$ ambos os simuladores usam **limite de passos**.

Engenharia e reprodutibilidade

- Python 3.10+, só biblioteca padrão (sem `pip install`).
- 32 testes (`python -m unittest discover -s testes`), incluindo o **oráculo**.
- Formalismo **explícito**: parser + β -redução; δ como relação + árvore — nada de "função pronta" devolvendo a resposta.
- Repositório com README, rastreamentos, saídas brutas e declaração de uso de IA.

Obrigado!

Equipe: Yuri Aguiar · João Rath · Pedro Paulo — CC5NA

Repositório: <https://github.com/yuriaguiardev/teoria-computabilidade>

Referências (resumo): Sipser (2013); Pierce (2002); Barendregt (1984); Diverio & Menezes (2011); Church (1936); Turing (1936). *(lista completa em [referencias.md](#))*

Perguntas?